

Week_4

Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

```
pavani@DESKTOP-380LEIN:~$ cd ~/ossp
pavani@DESKTOP-380LEIN:~/ossp$ pwd
/home/pavani/ossp
pavani@DESKTOP-380LEIN:~/ossp$ wait_waitpid.c
wait_waitpid.c: command not found
pavani@DESKTOP-380LEIN:~/ossp$ nano wait_waitpid.c
```

CODE:

```

GNU nano 8.7.1                                wait_waitpid.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    int i;
    pid_t pid;
    int status;
    printf("Parent Process PID: %d\n", getpid());
    for (i = 1; i <= 3; i++)
    {
        pid = fork();
        if (pid < 0)
        {
            perror("fork failed");
            return 1;
        }
        if (pid == 0)
        {
            printf("Child %d: PID = %d, Parent PID = %d\n", i, getpid(), getppid());
            sleep(i);
            printf("Child %d completed.\n", i);
            exit(i);
        }
    }
    printf("\nParent is waiting using wait()...\n");
    pid = wait(&status);
    if (pid > 0)
    {
        printf("wait(): Child with PID %d completed.\n", pid);
    }
    printf("\nParent is waiting using waitpid()...\n");
    for (i = 0; i < 2; i++)
    {
        pid = waitpid(-1, &status, 0);
        if (pid > 0)
        {
            printf("waitpid(): Child with PID %d completed.\n", pid);
        }
    }
    printf("\nAll child processes completed.\n");
    return 0;
}

```

OUTPUT:

```

pavani@DESKTOP-380LEIN:~/ossp$ nano wait_waitpid.c
pavani@DESKTOP-380LEIN:~/ossp$ gcc wait_waitpid.c -o wait_waitpid
pavani@DESKTOP-380LEIN:~/ossp$ ./wait_waitpid
Parent Process PID: 750
Child 1: PID = 751, Parent PID = 750
Child 2: PID = 752, Parent PID = 750

Parent is waiting using wait()...
Child 3: PID = 753, Parent PID = 750
Child 1 completed.
wait(): Child with PID 751 completed.

Parent is waiting using waitpid()...
Child 2 completed.
waitpid(): Child with PID 752 completed.
Child 3 completed.
waitpid(): Child with PID 753 completed.

All child processes completed.
pavani@DESKTOP-380LEIN:~/ossp$ █

```

Parent process completed.

REPORT:

I created a parent process that creates multiple child processes using `fork()`. I used `wait()` to wait for a child process to complete and used `waitpid()` to synchronize the remaining child processes. From the output, I understood that `wait()` can wait for any child process, while `waitpid()` gives more control over which child process is waited for.

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

pavani@DESKTOP-38OLEIN: ~/ossp

GNU nano 8.7.1

zombie.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());

        printf("Child is exiting...\n");

        exit(0);
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent is sleeping...\n");
        printf("Child will become a zombie.\n");

        sleep(30);

        printf("Parent completed.\n");
    }

    return 0;
}
```

Output:

```
pavani@DESKTOP-380LEIN:~/ossp$ nano zombie.c
pavani@DESKTOP-380LEIN:~/ossp$ gcc zombie.c -o zombie
pavani@DESKTOP-380LEIN:~/ossp$ ./zombie
Parent Process
Parent PID: 797
Child PID: 798
Parent is sleeping...
Child will become a zombie.
Child Process
Child PID: 798
Child is exiting...
Parent completed.
```

```
GNU nano 8.7.1                                zombie_fixed.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());

        printf("Child is exiting...\n");

        exit(0);
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        // Wait for the child to finish
        wait(NULL);

        printf("Child process completed.\n");
        printf("Zombie process eliminated.\n");
    }

    return 0;
}
```

OUTPUT:

```
pavani@DESKTOP-380LEIN:~/ossp$ gcc zombie_fixed.c -o zombie_fixed
pavani@DESKTOP-380LEIN:~/ossp$ ./zombie_fixed
Parent Process
Parent PID: 867
Child PID: 868
Child Process
Child PID: 868
Child is exiting...
Child process completed.
Zombie process eliminated.
pavani@DESKTOP-380LEIN:~/ossp$
```

REPORT:

I created a child process that becomes a zombie when the parent does not immediately collect its exit status. I checked the process table and observed the zombie process. Then I modified the program by using proper synchronization with `wait()` so that the parent collects the child process after it finishes. I understood that proper synchronization helps to prevent zombie processes.